

Manual Técnico do Script `app.py` - Sistema Hórus

Versão do Documento: 1.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python (Flask)

Este manual técnico descreve em detalhes o script `app.py`, que atua como o controlador principal da interface web do sistema Hórus. O `app.py` é responsável por gerenciar as rotas web, autenticação de usuários, uploads de arquivos, geração de relatórios e integrações com outros módulos do sistema. Ele segue o framework Flask para criar uma aplicação web MVC (Model-View-Controller), focando em segurança, modularidade e usabilidade.

O documento explica as funcionalidades principais, configurações, rotas disponíveis, restrições de uso e mecanismos de segurança implementados. Ele é destinado a administradores, desenvolvedores e mantenedores que precisam compreender ou modificar o comportamento da aplicação web.

1. Introdução e Propósito

O `app.py` é o núcleo da interface web do Hórus, uma plataforma de Threat Intelligence para processamento e análise de credenciais vazadas. Ele orquestra interações entre o usuário, o banco de dados PostgreSQL e outros scripts como `credenciais_processor.py` (processamento de dados), `statistics_dashboard.py` (geração de estatísticas), `password_analyzer.py` (análise de senhas) e `bttnng_integration.py` (integração com API externa).

O script gerencia sessões seguras, autenticação com hash de senhas (via Werkzeug), uploads de arquivos com verificação de duplicatas (hash MD5) e geração de relatórios em HTML/CSV. Ele opera em um ambiente seguro com SSL/TLS, headers de proteção e logs de auditoria para rastrear ações críticas.

1.1 Dependências e Integrações

- **Framework:** Flask (gerenciamento de rotas e sessões).
- **Banco de Dados:** PostgreSQL (via psycopg2), com adaptadores para autenticação (`auth`) e repositório de dados (`repo`).
- **Segurança:** Werkzeug para hash de senhas, Flask-Session para gerenciamento de sessões em filesystem.
- **Outras Bibliotecas:** Pandas para manipulação de dados em relatórios, Pandas para exportação CSV, SSL para contexto HTTPS.
- **Integrações Externas:** API BTTNG (sincronização manual/admin), Request Tracker (RT) via scripts externos.
- **Scripts Internos:** Chama funções de `credenciais_processor.py` para processamento de uploads, `statistics_dashboard.py` para dashboards, `password_analyzer.py` para análises de senhas e `bttng_integration.py` para sincronizações.

2. Funcionalidades Principais

O `app.py` fornece uma interface web completa para o Hórus, com foco em usabilidade e segurança. Abaixo, as principais funcionalidades:

- **Autenticação e Sessões:** Gerencia login/logout com verificação de credenciais hashadas. Sessões expiram em 30 minutos e são armazenadas em filesystem com permissões restritas (modo 0600). Registra IPs em logs de auditoria para tentativas de login.
- **Gerenciamento de Usuários (Admin):** Permite criar, reativar, desativar, excluir e alterar senhas de usuários. Restrições incluem proteção ao admin principal (ID 3) e prevenção de auto-modificação.
- **Upload de Arquivos:** Suporta múltiplos arquivos simultâneos (.txt, .xls, .xlsx) com limite de 100MB. Verifica duplicatas via MD5, processa via `process_file` e exibe resumo de resultados (novos/duplicados).
- **Busca e Relatórios:** Rotas para busca por usuário (parcial, case-insensitive), relatórios únicos/gerais com ordenação dinâmica e exportação CSV. Mascara senhas em visualizações.
- **Estatísticas e Análises:** Integra dashboards de estatísticas e análise de senhas, com gráficos em base64 e exportações CSV.
- **Inserção Manual:** Permite adicionar credenciais individualmente, com validações reutilizadas de outros scripts.
- **Sincronização Manual:** Rota admin para sincronizar dados da API BTTNG.
- **Auditoria e Logs:** Registra ações em banco (via `AuditLog`) e arquivos de log (access, error, security). Inclui logs para impressões e downloads.
- **Tratamento de Erros:** Handlers para 404/500 com templates personalizados.
- **Segurança Adicional:** Headers anti-XSS, anti-clickjacking; SSL com ciphersuites seguras e Perfect Forward Secrecy.

3. Configurações

O script define configurações globais e de ambiente via `app.config.update()` e variáveis de ambiente (`.env`):

- **Sessões:** Tipo 'filesystem', diretório '/tmp/flask_session', threshold 500 arquivos, refresh por request, lifetime 30 min, cookie HTTPOnly/SameSite=Lax (não Secure por default).
- **Uploads:** Pasta 'uploads', extensões permitidas {'txt', 'xls', 'xlsx'}, max 100MB.
- **Resultados:** Pasta 'resultados' para arquivos gerados.
- **Banco de Dados:** Usa `get_active_db_config()` para auth/repo; autocommit falso em conexões.
- **Globais:** `CURRENT_TIMESTAMP` e `CURRENT_USER` para templates; `CODIFICACAO='utf-8'`.
- **SSL:** Carrega certificados de '/opt/k/', ciphers ECDHE-*, TLS 1.2-1.3, opções para PFS.
- **Outras:** `JSON_AS_ASCII=False`, `MYSQL_DATABASE_CHARSET='utf8mb4'` (embora use PostgreSQL).

Restrições: Não usa cookies secure por default (`SESSION_COOKIE_SECURE=False`); `debug=True` em execução.

4. Rotas e Endpoints

O `app.py` define rotas com decoradores `@login_required` e `@admin_required` para controle de acesso. Cada rota é descrita abaixo, incluindo métodos, parâmetros, funcionalidades e restrições:

- **/ (GET):** Página inicial. Requer login. Renderiza 'index.html' com timestamp e username.
- **/documentacao (GET):** Exibe documentação técnica. Requer login. Renderiza 'Manual_tecnico.html'.
- **/login (GET/POST):** Formulário de login. Verifica hash de senha, atualiza `last_login`, registra IP em auditoria. Redireciona se já logado.
- **/logout (GET):** Limpa sessão e redireciona para login. Registra em auditoria.

- **/manual_insertion** (GET/POST): Inserção manual de credenciais. Valida campos, usa funções de validação externas, insere no banco com hash SHA1. Registra em auditoria se sucesso.
- **/change_password** (GET/POST): Troca de senha do usuário logado. Verifica senha atual, hash da nova.
- **/admin/users** (GET/POST): Gerenciamento de usuários (admin only). Lista todos (ativos/inativos), cria/reactiva usuários com hash de senha. Registra em auditoria.
- **/admin/users/<user_id>/enable** (POST): Reactiva usuário (admin only). Protege auto-modificação e admin principal. Registra em auditoria.
- **/admin/users/<user_id>/disable** (POST): Desativa usuário (soft delete, admin only). Mesmas proteções. Registra em auditoria.
- **/admin/users/<user_id>/delete** (POST): Exclui permanentemente (hard delete, admin only). Mesmas proteções. Registra em auditoria.
- **/admin/users/<user_id>/change-password** (POST): Altera senha de outro usuário (admin only). Requer JSON com new_password (>=6 chars). Protege admin principal.
- **/admin/sync-bttng** (POST): Sincronização manual BTTNG (admin only). Usa BTTNGIntegration, registra em auditoria.
- **/api/search_by_user** (GET): Busca API por usuário em 'credenciais' (parcial, ILIKE). Retorna JSON com registros.
- **/api/search_by_user_geral** (GET): Busca API por usuário em 'credenciais_geral'.
- **/estatisticas** (GET): Dashboard de estatísticas. Usa DashboardEstatisticas para dados e gráficos.
- **/relatorio** (GET): Relatório de credenciais únicas. Suporta ordenação (sort/direction), mascara senhas.
- **/delete_records** (POST): Arquiva e deleta registros selecionados. Requer senha de confirmação, transação atômica, registra em auditoria.
- **/relatorio_geral** (GET): Relatório geral de credenciais.
- **/get_report_geral** (GET): Conteúdo HTML do relatório geral.

- **/analise_senhas** (GET): Análise de senhas. Usa PasswordAnalyzer para resultados.
- **/download_relatorio_senhas** (GET): Exporta CSV de análise de senhas.
- **/download_relatorio/<report_type>** (GET): Exporta CSV de relatório (normal/geral). Registra em auditoria.
- **/upload** (POST): Upload de múltiplos arquivos. Verifica extensões, hash MD5, processa cada um, exibe resumo.
- **/search** (GET/POST): Busca por usuário via form. Registra em auditoria.
- **/log-print-action** (POST): Registra ação de impressão em auditoria (para relatórios/buscas).
- **Error Handlers**: 404 (página não encontrada), 500 (erro interno). Loga warnings/errors.

5. Restrições e Segurança

- **Acesso**: Rotas protegidas por login/admin. Sessões limitadas a 30 min. Admin principal (ID 3) tem privilégios exclusivos (gerenciar outros admins).
- **Uploads**: Apenas extensões permitidas, max 100MB, hash MD5 para deduplicação. Arquivos salvos em 'uploads'.
- **Dados Sensíveis**: Senhas mascaradas em visualizações/relatórios. Hashes SHA1 para armazenamento/comparação.
- **Auditoria**: Todas ações críticas (login, upload, deleção, etc.) registradas em banco e logs. Inclui IPs para logins.
- **Segurança Web**: Headers (STS, X-Content-Type-Options, etc.), SSL com TLS 1.2+, PFS. ProxyFix para X-Forwarded-For.
- **Restrições Gerais**: Não permite instalação de pacotes (sem internet). Debug=True (não para produção). Limites em queries para performance.
- **Erros**: Tratados com flashes, logs e templates personalizados. Transações com rollback em falhas.

Nota Final

O `app.py` é modular e extensível, mas alterações devem preservar segurança e conformidade (ex: LGPD). Consulte o código para detalhes ou integre novos módulos via blueprints. Em produção, desative debug e ative `SESSION_COOKIE_SECURE`.